

Lecture 10: Dependency Preservation and 3NF

Amit Kumar Dhar

August 22, 2025

A Problem with BCNF

BCNF Decomposition: A Motivating Example

Consider a relation that tracks which instructor teaches which course section for a student.

Schema: `Enroll(StudentID, CourseID, InstructorName)`

Functional Dependencies (FDs):

- An instructor teaches exactly one course:
 $\text{InstructorName} \rightarrow \text{CourseID}$
- A student in a specific course has only one instructor:
 $\{\text{StudentID}, \text{CourseID}\} \rightarrow \text{InstructorName}$

Candidate Key: $\{\text{StudentID}, \text{InstructorName}\}$

The relation is not in BCNF because of the FD $\text{InstructorName} \rightarrow \text{CourseID}$, where InstructorName is not a superkey.

Decomposing into BCNF

To satisfy BCNF, we decompose Enroll based on the violating FD ($\text{InstructorName} \rightarrow \text{CourseID}$):

1. **R1(InstructorName, CourseID)**
2. **R2(StudentID, InstructorName)**

This decomposition is in BCNF. But we have a problem...

$\{\text{StudentID}, \text{CourseID}\} \rightarrow \text{InstructorName}$ **is lost!**

- This FD cannot be checked without performing a join between R1 and R2, which is inefficient.
- We can now insert a student into multiple sections of the same course with different instructors, violating our original constraint.

Goal: A decomposition that is "good enough" but also preserves original dependencies.

Third Normal Form (3NF)

Definition of 3NF

A relation R is in 3NF if for every **non-trivial** functional dependency $\alpha \rightarrow \beta$, at least one of the following conditions holds:

1. α is a superkey for R. (This is the BCNF condition)
2. **OR**, each attribute in β is part of some candidate key of R (i.e., is a *prime attribute*).

This is a relaxation of the BCNF rule. It allows some dependencies where the determinant is not a superkey, as long as the determined attributes are "important" (part of a key).

3NF: The Motivating Example Revisited

Let's check our original Enroll relation against the 3NF definition.

Schema: Enroll(StudentID, CourseID, InstructorName)

FDs:

- InstructorName $\rightarrow$ CourseID
- {StudentID, CourseID} $\rightarrow$ InstructorName

Candidate Keys: {StudentID, InstructorName} and {StudentID, CourseID}

Let's check the violating FD: InstructorName $\rightarrow$ CourseID

1. Is InstructorName a superkey? **No.**
2. Is CourseID part of a candidate key? **Yes**, it is part of {StudentID, CourseID}.

The relation Enroll is already in **3NF**!

3NF Decomposition Algorithm

Canonical Cover for a Set of FDs

A **canonical cover** (F_c) for a set of FDs F is a simplified, minimal set of FDs that is equivalent to F . It has two main properties:

1. **No extraneous attributes:** For every FD $\alpha \rightarrow \beta$ in F_c , it is not possible to remove an attribute from α (the left side) or β (the right side) without changing the closure F^+ .
2. **No redundant FDs:** Each FD in F_c is essential and cannot be derived from the other FDs in F_c .

Think of it as the "cleanest" version of the FDs, which is a crucial starting point for the 3NF decomposition algorithm.

Example: Computing a Canonical Cover

Given FDs (F):

- $A \rightarrow BC$
- $B \rightarrow C$
- $A \rightarrow B$
- $AB \rightarrow C$

Step 1: Decompose FDs using the union rule.

- $F_1 = \{A \rightarrow B, A \rightarrow C, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$

Example: Computing a Canonical Cover

Given FDs (F):

- $A \rightarrow BC$
- $B \rightarrow C$
- $A \rightarrow B$
- $AB \rightarrow C$

Step 1: Decompose FDs using the union rule.

Step 2: Remove extraneous attributes from the left side.

- In $AB \rightarrow C$, is B extraneous? We check if $A \rightarrow C$ follows from the original FDs. Yes, it does ($A \rightarrow B$ and $B \rightarrow C$). So, we simplify $AB \rightarrow C$ to $A \rightarrow C$.
- $F_2 = \{A \rightarrow B, A \rightarrow C, B \rightarrow C\}$

Example: Computing a Canonical Cover

Given FDs (F):

- $A \rightarrow BC$
- $B \rightarrow C$
- $A \rightarrow B$
- $AB \rightarrow C$

Step 1: Decompose FDs using the union rule.

Step 2: Remove extraneous attributes from the left side.

Step 3: Remove redundant FDs.

- Is $A \rightarrow C$ redundant? Can we derive it from the rest of F_2 ?
Yes, from $A \rightarrow B$ and $B \rightarrow C$. So, we remove it.
- $F_3 = \{A \rightarrow B, B \rightarrow C\}$

Example: Computing a Canonical Cover

Given FDs (F):

- $A \rightarrow BC$
- $B \rightarrow C$
- $A \rightarrow B$
- $AB \rightarrow C$

Step 1: Decompose FDs using the union rule.

Step 2: Remove extraneous attributes from the left side.

Step 3: Remove redundant FDs.

Final Canonical Cover (F_c): $\{A \rightarrow B, B \rightarrow C\}$

Algorithm for 3NF Decomposition

This algorithm guarantees a **dependency-preserving** and **lossless-join** decomposition into 3NF.

1. Find a **canonical cover** F_c for the set of FDs F .
2. For each FD $\alpha \rightarrow \beta$ in F_c , create a new relation with the schema $(\alpha \cup \beta)$.
3. Check if any of the newly created relations contains a candidate key for the original relation R .
4. If not, add one more relation to the decomposition whose schema consists of just the attributes of a candidate key for R .

3NF Decomposition: Example 1

Schema:

R(StudentID, Name, Street, City, Zip, CourseID, Grade)

FDs (F):

- StudentID $\rightarrow$ Name, Street, City, Zip
- Zip $\rightarrow$ City
- {StudentID, CourseID} $\rightarrow$ Grade

1. Find Canonical Cover (F_c):

- StudentID $\rightarrow$ Name, Street, Zip (Removed City by transitivity)
- Zip $\rightarrow$ City
- {StudentID, CourseID} $\rightarrow$ Grade

3NF Decomposition: Example 1

1. Find Canonical Cover (F_c):

- $\text{StudentID} \rightarrow \text{Name}, \text{Street}, \text{Zip}$ (Removed City by transitivity)
- $\text{Zip} \rightarrow \text{City}$
- $\{\text{StudentID}, \text{CourseID}\} \rightarrow \text{Grade}$

2. Create Relations from F_c :

- $R1(\underline{\text{StudentID}}, \text{Name}, \text{Street}, \text{Zip})$
- $R2(\underline{\text{Zip}}, \text{City})$
- $R3(\underline{\text{StudentID}}, \underline{\text{CourseID}}, \text{Grade})$

3. Check for Candidate Key: The candidate key for R is $\{\text{StudentID}, \text{CourseID}\}$. R3 contains this key, so we are done.

Final Decomposition: R1, R2, R3.

3NF Decomposition: Example 2

Schema: $R(A, B, C, D, E)$

FDs (F):

- $A \rightarrow BC$
- $C \rightarrow D$
- $B \rightarrow E$
- $E \rightarrow A$

1. Canonical Cover (F_c): The given set is already minimal.

$$F_c = \{A \rightarrow B, A \rightarrow C, C \rightarrow D, B \rightarrow E, E \rightarrow A\}$$

3NF Decomposition: Example 2

1. Canonical Cover (F_c): The given set is already minimal.

$$F_c = \{A \rightarrow B, A \rightarrow C, C \rightarrow D, B \rightarrow E, E \rightarrow A\}$$

2. Create Relations from F_c :

- $R1(\underline{A}, B)$
- $R2(\underline{A}, C)$
- $R3(\underline{C}, D)$
- $R4(\underline{B}, E)$
- $R5(\underline{E}, A)$

3. Check for Candidate Key: The candidate keys for R are $\{A\}$, $\{B\}$, and $\{E\}$.

- $R1$ contains the key $\{A\}$.
- $R4$ contains the key $\{B\}$.
- $R5$ contains the key $\{E\}$.

Since a candidate key is present in at least one of the relations, the decomposition is complete.

Thank You!
Questions?